OpenVLA · arXiv 2406.09246 · CoRL 2024

Empiricist

11+3 · seat 1 = Kanta · BUILD FIRST

LoRA ladder + AR vs DP/FM densest
Taxonomy · tattoos · P0–P2 · hypotheses

Oct 21 paper club · VLA Architectures

Spine: naïve discrete AR @ ~5–6 Hz · historical open AR baseline

BEHAVIOR deploy = $\pi_{0.5}$ / OpenPI / GR00T · OpenVLA = open AR control

Action-family taxonomy · CLUB SPINE

Must appear in every role · no-chunk AR ≈ 5–6 Hz

Family	Model	Signature
Naïve discrete AR bins	**OpenVLA / RT-2**	Lang grounding @ ~5–6 Hz; weak high-Hz
DDPM chunks	**Octo / DP**	Smoother narrow skills
FM chunks	**π_0 / $\pi_{0.5}$**	2025–26 deploy winners
Compressed discrete AR	**FAST / OpenVLA+FAST / π_0-FAST**	AR recovers high-Hz

Syllabus: OpenVLA = historical open AR baseline · BEHAVIOR deploy = $\pi_{0.5}$ / OpenPI / GR00T

Tattoo board · Empiricist must-cite numbers

Say aloud early · then never lose them

WidowX Bridge

70.6%

vs RT-2-X 50.6 · Octo 20.0

Google robot

85.0%

vs RT-2-X 78.3 · Octo 26.7

Headline

+16.5 pp

vs RT-2-X 55B · 7× smaller

LoRA r=32

68.2%

≈ full FT 69.7 (~1.4% params)

LIBERO FT avg

76.5%

rank 1.5 > Octo 75.1 > DP 72.4

int4 serve

71.9%

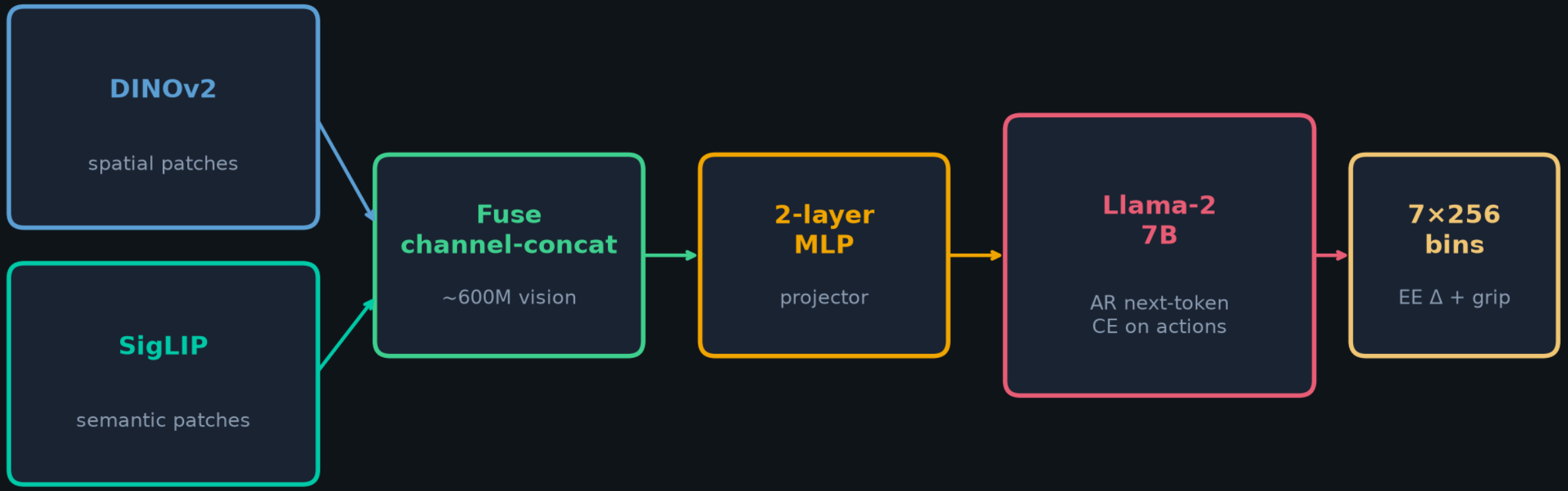
~7.0 GB ≈ bf16 71.3%

Also tattoo: frozen vision 47% · Bridge-only −30 pp · infer ~6 Hz / 4090 bf16

Do NOT claim +16.5 pp or Bridge parity from Partial LIBERO smoke alone

OpenVLA architecture · Prismatic VLM → discrete AR actions

No wrist / proprio / history / chunks · single 3rd-person RGB 224^2



Prompt: “What should the robot do to {task}? A:” + 7 action tokens

Train: OXE ~970k · 27 epochs · 64×A100 ×14d ≈ 21.5k A100-h · LR 2e-5 · vision UNFROZEN

Reproduction ladder · Empiricist build order

HF smoke → LoRA r=32 → DP-matched side-by-side → optional FAST+

Scout success = working ckpt + LoRA LIBERO direction + honest Hz/VRAM logs

NOT Bridge 170-rollout parity · env: /workspace/openvla-quiet/env_outline/

Priority table · P0 / P1 / P2 / Defer

Pri	Experiment	Smoke
P0	HF ckpt dry infer (~15–17 GB bf16)	Y
P0	int4 / int8 serve (~7 GB int4)	Y
P0	LIBERO-Spatial LoRA r=32 scout	Partial
P1	LoRA r sweep · full FT vs LoRA · vision FT vs freeze	Y / Partial
P1	AR discrete vs DP · OpenVLA vs π_0 FM vs Octo @ matched commit_s	Partial
P2	no-op filter · FAST drop-in vs 256 bins	Y / Partial
Defer	21.5k A100-h OpenX re-pretrain · multi-robot real suites	N

Hypotheses · falsifiable Empiricist claims

H-lora

LoRA r=32 ≈ full FT within ~2 pp

at ~1.4% trainable params

H-ar-lang

AR wins multi-object language

DP/FM win precision / smooth Hz

H-vision

Unfreeze vision helps

frozen often collapses / 47% FT

H-quant

int4 ≈ bf16 task SR

int8 fails mainly via latency

H-hz

~6 Hz is a control bottleneck

gaps track Hz as much as representation

H-hygiene

No-op filter necessary for AR

without it OpenVLA freezes (App E)

Falsifiers: LoRA ≪ FT by ≫5 pp · DP dominates every suite · int4 collapses tokens · freeze ties

Log (ckpt, prec, lora_r, suite, steps, Hz, VRAM) beside every success

AR discrete vs DP / FM continuous · Empiricist contrast

Axis	OpenVLA AR	DP / Octo	FM (π_0 family)
Action	7×256 bin ids	Denoise chunk	Flow on chunk
Temporal	Single step NO chunk	Chunk + TE/receding	Chunk + receding
Loss	Token CE	Score / ε-pred	FM / velocity
Infer cost	7 AR × 7B LM	K steps × small head	~10 NFE × ~300M
Hz	~6 (4090 bf16)	5–15+ w/ chunks	~30 challenge
Lang	STRONG (VLM NTP)	Needs lang encoder	VLM + task emb
Dexterity	Grid / choppy if slow	STRONG	STRONG
2025–26 role	Open ancestor / lang FT	Strong IL baseline	DEFAULT serve

Pick: open lang FT → OpenVLA LoRA · smooth high-Hz → DP/FM · BEHAVIOR'26 → $\pi_{0.5}$ + optional FAST aux

PEFT / LoRA table · Table 1 moral

Method	Success	Trainable M	Peak mem
Full FT	69.7 ± 7.2%	7188.1	163.3 GB*
Last layer only	30.3 ± 6.1%	465.1	51.4 GB
Frozen vision	47.0 ± 6.9%	6760.4	156.2 GB*
Sandwich	62.1 ± 7.9%	914.2	64.0 GB
LoRA r=32 ★	**68.2 ± 7.5%**	**97.6 (1.4%)**	**59.7 GB**
LoRA r=64	68.2 ± 7.8%	195.2	60.5 GB

Default r=32 · 10–15 h on 1×A100 · ~8× compute cut vs full FT on 8×A100

LIBERO sim · Table 12 · LoRA FT

Method	Spatial	Object	Goal	Long	Avg / Rank
DP scratch	78.3	92.5 ★	68.3	50.5	72.4 / 2.5
Octo FT	78.9	85.7	84.6 ★	51.1	75.1 / 2
OpenVLA LoRA	84.7 ★	88.4	79.2	53.7 ★	76.5 / 1.5

Pattern: OpenVLA wins Spatial/Long avg rank · DP wins Object · Octo wins Goal

Hygiene: drop no-ops · filter fails · 180° rotate · third-person · regen 256→224

Thin margins — do not treat as decisive VLA superiority

GPU honesty · Empiricist budget

Workload	VRAM	Wall	Smoke
Infer bf16 smoke	**15–18 GB**	minutes	**Y**
Infer int4	**7–10 GB**	minutes	**Y**
LoRA r=32 LIBERO scout	**24–48 GB (aim)**	10–20 h	**Partial**
AR vs DP matched suite	**24–48 GB**	1–2 days	**Partial**
OpenX re-pretrain	**64×A100**	14 days	**N / Defer**

Quiet P0–P1 budget target ≲40–80 GPU-h · exit nonzero if CUDA claimed but unavailable

Never call CPU-only a “GPU OpenVLA run”

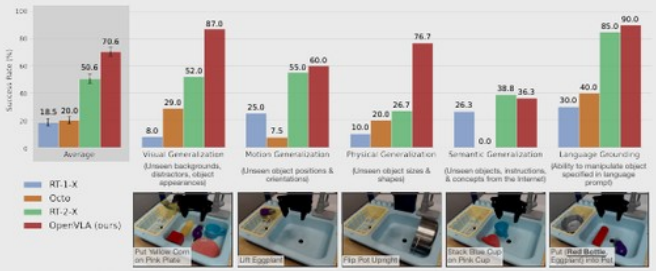


Figure 3: **BridgeData V2 WidowX robot evaluation tasks and results.** We evaluate OpenVLA and prior state-of-the-art generalist robot policies on a comprehensive suite of tasks covering several axes of generalization, as well as tasks that specifically assess language conditioning ability. OpenVLA achieves highest overall performance and even outperforms closed-source model RT-2-X in all categories except for semantic generalization. Average success rates ± StdErr are computed across 170 total rollouts per approach. See Table 4 for detailed results.

techniques for large transformer model training such as automatic mixed precision (AMP, PyTorch [75]), FlashAttention [76], and fully sharded data parallelism (FSDP, Zhao et al. [77]). Out of the box, the OpenVLA codebase has full support for training on the Open X dataset, integrates with HuggingFace's [21] `AutoModel` class, and supports LoRA fine-tuning [26] and quantized model inference [27, 88].

5 Experiments

The goal of our experimental evaluations is to test OpenVLA's ability to serve as a powerful multi-robot control policy out of the box, as well as be a good initialization for fine-tuning to new robot tasks. Concretely, we aim to answer the following questions:

1. How does OpenVLA compare to prior generalist robot policies, when evaluating on multiple robots and various types of generalization?
2. Can OpenVLA be effectively fine-tuned on a new robot setup and task, and how does it compare to state-of-the-art data-efficient imitation learning approaches?
3. Can we use parameter-efficient fine-tuning and quantization to reduce the computational requirements for training and inference of OpenVLA models and make them more accessible? What are the performance-compute trade-offs?

5.1 Direct Evaluations on Multiple Robot Platforms

Robot Setups and Tasks. We evaluate OpenVLA's performance "out-of-the-box" on two robot embodiments: the WidowX robot from the BridgeData V2 evaluations [6] (see Fig. 1, left) and the mobile manipulation robot from the RT-1 and RT-2 evaluations [2, 7] ("Google robot"; see Fig. 1, middle). Both platforms have been extensively used in prior works for evaluating generalist robot policies [1, 2, 5, 7]. We define a comprehensive set of evaluation tasks in each environment that covers various axes of generalization, such as **visual** (unseen backgrounds, distractor objects, colors/appearances of objects); **motion** (unseen object positions/orientations); **physical** (unseen object sizes/shapes); and **semantic** (unseen target objects, instructions, and concepts from the Internet) generalization. We also assess language conditioning ability in scenes with multiple objects, testing whether the policy can manipulate the correct target object, as specified in the user's prompt. See bottom row of Fig. 3 and Fig. 4 for example task images in the BridgeData V2 and Google robot evaluations, respectively. Overall, we evaluated each method in 170 rollouts (17 tasks with 10 trials each) for BridgeData V2 experiments and 60 rollouts (12 tasks with 5 trials each) for Google robot experiments. A detailed breakdown of all tasks and how they differ from the training data is in

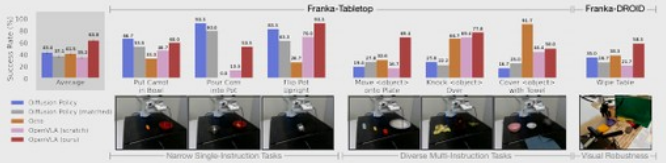


Figure 5: **Adapting to new robot setups.** We evaluate the state-of-the-art Diffusion Policy trained from scratch on seven Franka Emika Panda tasks (10–150 demonstrations each), as well as generalist robot policies Octo and OpenVLA fine-tuned on the same data. Diffusion Policy exhibits strong performance on narrow single-instruction tasks, while Octo and OpenVLA perform better on diverse fine-tuning tasks involving multiple instructions and distractor objects. Overall, OpenVLA achieves highest aggregate performance across both setups, suggesting that it is an effective default for learning a policy on a downstream task. Average success rates ± StdErr are computed across 129 rollouts per approach (99 for Franka-Tabletop tasks and 30 for Franka-DROID tasks). See Table 7 for detailed results.

mounted on a movable standing desk. The setups use 5Hz and 15 Hz non-blocking controllers, respectively. We choose Franka robot arms as the target embodiment for our fine-tuning experiments since they are widely used in the robot learning community and thus a likely "target" of OpenVLA fine-tuning. We test on setups with different control frequencies to test OpenVLA's applicability to a range of use cases.

Comparisons. We compare to **Diffusion Policy** [3], a state-of-the-art data-efficient imitation learning approach, trained from scratch. We also compare to **Diffusion Policy (matched)**, a version of Diffusion Policy that matches the input and output specifications of OpenVLA.[3] Additionally, we evaluate **Octo** [5] fine-tuned on the target dataset, since it is currently the best generalist policy that supports fine-tuning (fine-tuning of RT-2-X is not supported through its inference API). We also fine-tune OpenVLA on the same target dataset, and the resulting policy is denoted by **OpenVLA**. Finally, as an ablation experiment, we compare to **OpenVLA (scratch)**, where we directly fine-tune the underlying base Prismatic VLM on the target robot setup – rather than fine-tuning the OpenX-pretrained OpenVLA model – to assess the benefit of large-scale robot pretraining.

We present the results in Fig. 5 (per-task breakdown in Appendix, Table 7). We find that both versions of Diffusion Policy are competitive with or outperform the generalist policies Octo and OpenVLA on narrower single-instruction tasks like "Put Carrot in Bowl" and "Pour Corn into Pot", but the pretrained generalist policies perform better in more diverse fine-tuning tasks that involve multiple objects in the scene and require language conditioning. OpenX pretraining for Octo and OpenVLA enables the models to better adapt to these more diverse tasks where language grounding is important; we see evidence for this in the lower performance of OpenVLA (scratch).

Overall, we find that OpenVLA achieves the highest average performance. Notably, most prior works achieve strong performance only in *either* narrow single-instruction *or* diverse multi-instruction tasks, resulting in widely varying success rates. OpenVLA is the only approach that achieves at least 50% success rate across all tested tasks, suggesting that it can be a strong default option for imitation learning tasks, particularly if they involve a diverse set of language instructions. For narrower but highly dexterous tasks, Diffusion Policy still shows smoother and more precise trajectories; incorporating action chunking and temporal smoothing, as implemented in Diffusion Policy, may help OpenVLA attain the same level of dexterity and may be a promising direction for future work (see Section 6 for a detailed discussion of current limitations).

[3]The full Diffusion Policy uses a two-step observation history with both images and proprioceptive state, and performs receding horizon control by predicting a chunk of T future actions and executing the first X actions in open-loop fashion before predicting the next chunk (for 15Hz control, we set $T = 16$, $X = 8$ like in the DROID prior work [11]; for 5Hz control, we reduce the chunk sizes to $T = 8$, $X = 3$). It is also the only method in Section 5.2 that predicts *absolute* Cartesian coordinates to control the robot; all other methods use *relative* position control. Diffusion Policy (matched) uses a single image as input, has no proprioceptive information and no observation history, and predicts a single relative position control action without action chunking.

5.3 Parameter-Efficient Fine-Tuning

The full fine-tuning runs of OpenVLA in the previous section used 8 A100 GPUs for 5-15 hours per task (depending on the dataset size) to achieve high performance. While this is substantially less compute than what is required for VLA pretraining, in this section we explore even more compute- and parameter-efficient fine-tuning approaches and investigate their effectiveness.

Concretely, we compare the following fine-tuning approaches: **full fine-tuning** updates all weights during fine-tuning, as described in Section 5.2; **last layer only** fine-tunes only the last layer of OpenVLA's transformer backbone and the token embedding matrix; **frozen vision** freezes the vision encoder but fine-tunes all other weights; **sandwich fine-tuning** unfreezes the vision encoder, token embedding matrix, and last layer; and **LoRA** uses the popular low-rank adaptation technique of Hu et al. [26] with multiple rank values r, applied to all linear layers of the model.

Table 1: **Parameter-efficient fine-tuning evaluation.** LoRA fine-tuning achieves the best performance-compute trade-off, matching full fine-tuning performance while training only 1.4% of the model parameters. Mean success ± StdErr computed across 33 rollouts per approach on select Franka-Tabletop tasks (see Table 8 for details). *: Sharded across 2 GPUs with FSDP [77].

Strategy	Success Rate	Train Params ($\times 10^6$)	VRAM (batch 16)
Full FT	**69.7 ± 7.2 %**	7,188.1	163.3 GB*
Last layer only	30.3 ± 6.1 %	465.1	51.4 GB
Frozen vision	47.0 ± 6.9 %	6,760.4	156.2 GB*
Sandwich	62.1 ± 7.9 %	914.2	64.0 GB
LoRA, rank=32	**68.2 ± 7.5%**	**97.6**	**59.7 GB**
rank=64	**68.2 ± 7.8%**	195.2	60.5 GB

We report fine-tuning success rates across multiple Franka-Tabletop tasks, as well as training parameter count and GPU memory requirements, in Table 1.[4] We find that only fine-tuning the network's last layer or freezing the vision encoder leads to poor performance, suggesting that further adaptation of the visual features to the target scene is crucial. In contrast, "sandwich fine-tuning" achieves better performance since it fine-tunes the vision encoder, and it consumes less GPU memory since it does not fine-tune the full LLM backbone. Lastly, LoRA achieves the best trade-off between performance and training memory consumption, outperforming "sandwich fine-tuning" and matching full fine-tuning performance while fine-tuning only 1.4% of the parameters. We find that the LoRA rank has negligible effect on policy performance and thus recommend using a default rank of $r = 32$. With LoRA, we can fine-tune OpenVLA on a new task within 10-15 hours on a *single* A100 GPU – an 8x reduction in compute compared to full fine-tuning.

5.4 Memory-Efficient Inference via Quantization

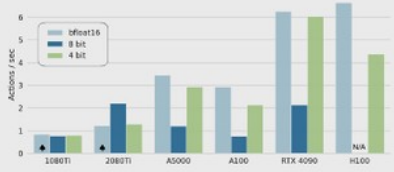


Figure 6: **OpenVLA inference speed for various GPUs.** Both bfloat16 and int4 quantization achieve high throughput, especially on GPUs with Ada Lovelace architecture (RTX 4090, H100). Further speed-ups are possible with modern LLM inference frameworks like TensorRT-LLM [89]. ♠: Model sharded across two GPUs to fit.

Precision	Bridge Success	VRAM
bfloat16	71.3 ± 4.8%	16.8 GB
int8	58.1 ± 5.1%	10.2 GB
int4	71.9 ± 4.7%	7.0 GB

Table 2: **Performance with quantized inference.** 4-bit quantization matches the performance of bfloat16 inference (our default approach) while reducing the GPU memory footprint by more than half. Mean success ± StdErr computed across 8 representative BridgeData V2 tasks [6] and 80 rollouts per approach (see Table 5 for details).

OpenVLA, a 7B-parameter model, consumes more memory at inference time than prior open-source generalist policies such as Octo, which has <100M parameters. We follow best-practices from LLM serving by saving and loading OpenVLA in `bfloat16` precision for inference (our default approach), which cuts the memory footprint in half, allowing us to serve OpenVLA on GPUs with only 16GB

[4] In Section 5.3 and Section 5.4, we experiment with a version of the OpenVLA model that is pretrained with a smaller robot data mixture (the same OpenX dataset mixture as Octo) and has a slightly smaller architecture which only uses a SigLIP [79] vision backbone instead of the fused DinoSigLIP encoder. We find that this simpler architecture still achieves strong performance in both fine-tuning tasks and "out-of-the-box" tasks.

Empiricist · close

P0: HF smoke + int4 + LoRA r=32 scout. Then DP-matched + optional FAST+.

H: LoRA≈full FT ~2 pp · AR wins multi-object lang · DP/FM win precision/Hz

Q&A · Oct 21 · OpenVLA / 2406.09246

Action-family taxonomy · CLUB SPINE

Must appear in every role · no-chunk AR ≈ 5–6 Hz

Family	Model	Signature
Naïve discrete AR bins	**OpenVLA / RT-2**	Lang grounding @ ~5–6 Hz; weak high-Hz
DDPM chunks	**Octo / DP**	Smoother narrow skills
FM chunks	**π_0 / $\pi_{0.5}$**	2025–26 deploy winners
Compressed discrete AR	**FAST / OpenVLA+FAST / π_0-FAST**	AR recovers high-Hz

Syllabus: OpenVLA = historical open AR baseline · BEHAVIOR deploy = $\pi_{0.5}$ / OpenPI / GR00T